

第十一章

SQL优化

digoal

目标

- 了解如何优化SQL。
- 熟悉一些常见SQL优化手段。

目录

- 如何分析SQL性能
- SQL优化举例

性能分析 - explain

- <https://www.postgresql.org/docs/10/static/sql-explain.html>
- https://github.com/digoal/blog/blob/master/201704/20170424_06.md
- HDB PG
- https://github.com/digoal/blog/blob/master/201712/20171204_02.md

设置需要TRACE的开关。
查看详细的输出，逐个分析。
最重要的是分析
explain
(analyze,verbose,timing,costs,buffers
的结果)

```
Command:      EXPLAIN
Description: show the execution plan of a statement
Syntax:
EXPLAIN [ ( option [, ...] ) ] statement
EXPLAIN [ ANALYZE ] [ VERBOSE ] statement

where option can be one of:

    ANALYZE [ boolean ]
    VERBOSE [ boolean ]
    COSTS [ boolean ]
    SETTINGS [ boolean ]
    BUFFERS [ boolean ]
    TIMING [ boolean ]
    SUMMARY [ boolean ]
    FORMAT { TEXT | XML | JSON | YAML }

URL: https://www.postgresql.org/docs/devel/sql-explain.html
```

性能分析 - explain

- set client_min_messages=debug5;
- set log_checkpoints = on;
- set log_error_verbosity = verbose ;
- set log_lock_waits = on;
- set log_replication_commands = off;
- set log_temp_files = 0;
- set track_activities = on;
- set track_counts = on;
- set track_io_timing = on;
- set track_functions = 'all';
- set trace_sort=on;
- set log_statement_stats = off;
- set log_parser_stats = on;
- set log_planner_stats = on;
- set log_executor_stats = on;
- set log_autovacuum_min_duration=0;
- set deadlock_timeout = '1s';
- set debug_print_parse = off;
- set debug_print_rewritten = off;
- set debug_print_plan = off;
- set debug_pretty_print = on;
- explain (analyze,verbose,timing, costs, buffers) select count(*),relkind from pg_class group by relkind order by count(*) desc limit 1;

设置需要TRACE的开关。
查看详细的输出，逐个分析。
最重要的是分析explain
(analyze,verbose,timing, costs, buffers的结果)

例子1

- postgres=# **explain (analyze,verbose,timing, costs,buffers,SETTINGS,summary)** select * from tbl123 where id in (select id from tbl1234);
- Hash Semi Join (cost=2693.00..213387.71 rows=100000 width=37) (actual time=24.507..**2460.292** rows=**100000** loops=1)
 - Output: tbl123.id, tbl123.info
 - Hash Cond: (tbl123.id = tbl1234.id)
 - Buffers: shared hit=83777
- -> **Seq Scan on public.tbl123** (cost=0.00..183332.58 rows=9999858 width=37) (actual time=0.008..**1079.710** rows=**10000000** loops=1)
 - Output: tbl123.id, tbl123.info
 - Buffers: shared hit=83334
- -> Hash (cost=1443.00..1443.00 rows=100000 width=4) (actual time=24.340..24.340 rows=100000 loops=1)
 - Output: tbl1234.id
 - Buckets: 131072 Batches: 1 Memory Usage: 4540kB
 - Buffers: shared hit=443
- -> Seq Scan on public.tbl1234 (cost=0.00..1443.00 rows=100000 width=4) (actual time=0.011..9.676 rows=100000 loops=1)
 - Output: tbl1234.id
 - Buffers: shared hit=443
- **Settings: effective_cache_size = '400GB', effective_io_concurrency = '0', enable_bitmapscan = 'off', enable_indexscan = 'off', enable_partitionwise_aggregate = 'on', enable_partitionwise_join = 'on', max_parallel_workers = '64', max_parallel_workers_per_gather = '0', parallel_leader_participation = 'off', random_page_cost = '1.1', work_mem = '64MB'**
- **Planning Time: 0.172 ms**
- **Execution Time: 2465.570 ms**
- (17 rows)

- postgres=# set enable_indexscan =on;
- postgres=# explain (analyze,verbose,timing, costs,buffers,SETTINGS,summary) select * from tbl123 where id in (select id from tbl1234);
- Merge Semi Join (cost=9752.76..14145.01 rows=100000 width=37) (actual time=23.086..79.061 rows=100000 loops=1)
 - Output: tbl123.id, tbl123.info
 - Merge Cond: (tbl123.id = tbl1234.id)
 - Buffers: shared hit=1553
 - -> Index Scan using tbl123_pkey on public.tbl123 (cost=0.43..263495.51 rows=9999858 width=37) (actual time=0.009..17.258 rows=100001 loops=1)
 - Output: tbl123.id, tbl123.info
 - Buffers: shared hit=1110
 - -> Sort (cost=9747.82..9997.82 rows=100000 width=4) (actual time=23.068..29.675 rows=100000 loops=1)
 - Output: tbl1234.id
 - Sort Key: tbl1234.id
 - Sort Method: quicksort Memory: 7760kB
 - Buffers: shared hit=443
 - -> Seq Scan on public.tbl1234 (cost=0.00..1443.00 rows=100000 width=4) (actual time=0.010..9.980 rows=100000 loops=1)
 - Output: tbl1234.id
 - Buffers: shared hit=443
- Settings: effective_cache_size = '400GB', effective_io_concurrency = '0', enable_bitmapsan = 'off', enable_partitionwise_aggregate = 'on', enable_partitionwise_join = 'on', max_parallel_workers = '64', max_parallel_workers_per_gather = '0', parallel_leader_participation = 'off', random_page_cost = '1.1', work_mem = '64MB'
- Planning Time: 0.179 ms
- Execution Time: **86.241 ms**
- (18 rows)

目录

- 如何分析SQL性能
- SQL优化举例

SQL优化举例

- = any in
- 模糊查询
- 相似查询优化
- GIS、时空 优化
- 分组TOP
- 递归、子查询优化
- IO与CPU放大
- correlate
- partial index 优化
- range 智能dns优化
- 导购系统优化
- count (distinct) 优化
count_distinct
- 分页优化
- 数据采样优化
- 近似统计优化
- 随机offset 优化
- 秒杀优化
- UUID IO优化
- 时序优化
- 索引与nulls 优化
- 任意字段组合查询优化
- 高并发事务优化
- 批量写入优化 extend,
vacuum,
- json 查询优化
- 标签筛选优化 (rum,
varbitx)
- 批量更新优化 (skip
lock, parallel)
- 复杂计算优化 (并行,
HASH, JOIN顺序,
HINT)

= any(array()) | in

- https://github.com/digoal/blog/blob/master/201801/20180131_02.md

模糊查询

- https://github.com/digoal/blog/blob/master/201605/20160506_02.md

相似查询

- https://github.com/digoal/blog/blob/master/201809/20180904_01.md
- https://github.com/digoal/blog/blob/master/201802/20180202_01.md
- https://github.com/digoal/blog/blob/master/201802/20180205_03.md

GIS优化

- https://github.com/digoal/blog/blob/master/201308/20130806_01.md

分组TOP

- https://github.com/digoal/blog/blob/master/201608/20160815_04.md

递归、子查询优化

- https://github.com/digoal/blog/blob/master/201612/20161201_01.md

IO与CPU放大

- https://github.com/digoal/blog/blob/master/201710/20171004_01.md
- https://github.com/digoal/blog/blob/master/201710/20171005_01.md

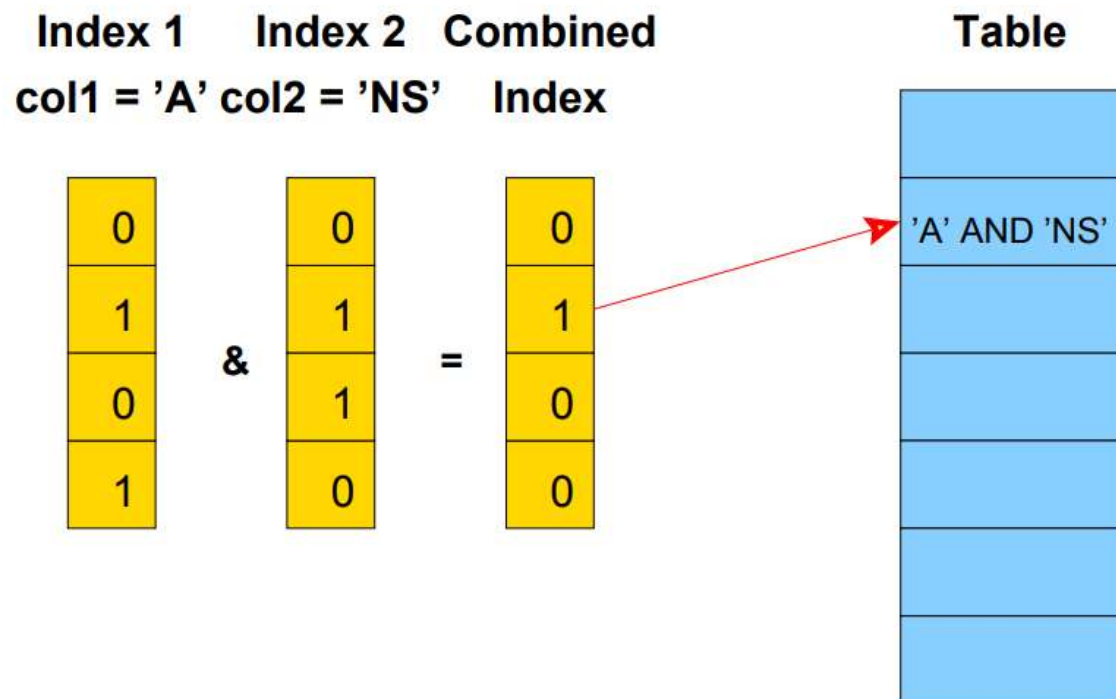
correlate、cluster

- 相关性
- IO放大
- CPU放大
- index [only] scan
 - 根据叶子节点顺序扫
 - 同一个HEAP PAGE可能进出很多遍shared buffer，小内存，大量扫描，可能导致较多物理IO。
- bitmap scan
 - HEAP block id 顺序扫描（每个BLOCK扫一遍）
 - heap recheck
 - recheck()
- 索引机制引入放大
 - GIST bound box
- 索引未完全命中情况
 - removed by filter
 - index cond
 - lossy index

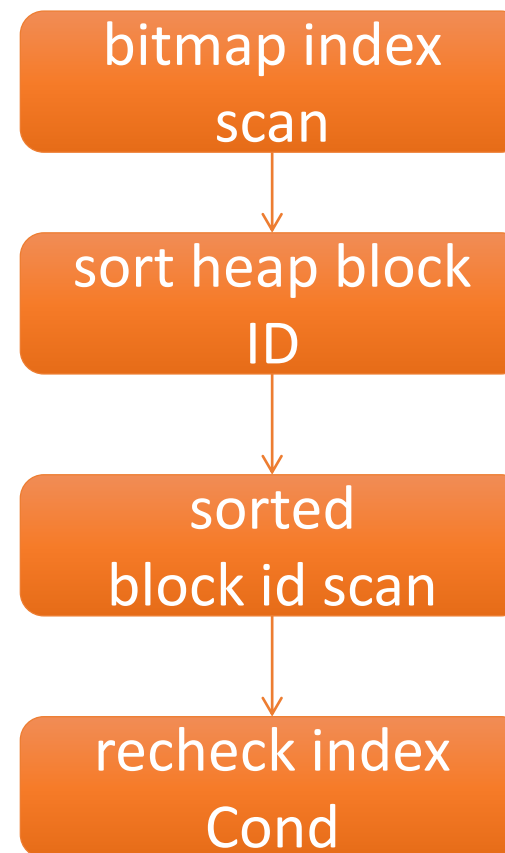
bitmap scan

1、multi-index combine **OR** internal combine(GIN)

Bitmap Scan



2、消除离散、重复读HEAP



bitmap scan

- index scan -> sort heap blockid -> scan heap block -> recheck index Cond.
- 优化器参考指标，相关性
- IO 放大问题消除
- SQL例子
 - https://github.com/digoal/blog/blob/master/201804/20180402_01.md

bitmap scan

```
-[ RECORD 2 ]-----+-----  
schemaname      | public  
tablename       | corr_test  
attname         | c2  
inherited       | f  
null_frac       | 0  
avg_width       | 4  
n_distinct      | -0.51138  
most_common_vals | {426318,766194,85!  
most_common_freqs | {6.66667e-05,6.66!  
histogram_bounds | {271,106225,20156!  
correlation     | 0.00410469 # 线性  
most_common_elems |  
most_common_elem_freqs |  
elem_count_histogram |
```

bitmap scan

- 离散扫描，每个BLOCK几乎都被重复扫描了140次，一个BLOCK刚好存储140条记录，说明这140条记录在顺序上完全离散。
- postgres=# explain (analyze,verbose,timing,costs,buffers) select * from corr_test where c2 between 1 and 10000000;
- QUERY PLAN
- -----
- Index Scan using idx_corr_test_2 on public.corr_test (cost=0.43..36296.14 rows=50000 width=8) (actual time=0.029..6563.525 rows=9999999 loops=1)
- Output: c1, c2
- Index Cond: ((corr_test.c2 >= 1) AND (corr_test.c2 <= 10000000))
- Buffers: shared hit=10027095
- Planning time: 0.089 ms
- Execution time: 7421.801 ms
- (6 rows)

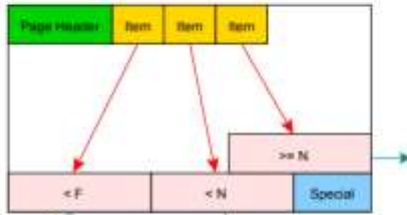
bitmap scan

- 使用位图扫描，位图扫描的原理是从索引中得到**HEAP BLOCK ID**，然后按**HEAP BLOCK ID** 排序后顺序扫描。
- postgres=# explain (analyze,verbose,timing, costs,buffers) select * from corr_test where c2 between 1 and 10000000;
- QUERY PLAN
- -----
- Bitmap Heap Scan on public.corr_test (cost=2844700.76..3038949.24 rows=10000032 width=8) (actual time=688.150..1939.259 rows=9999998 loops=1)
- Output: c1, c2
- Recheck Cond: ((corr_test.c2 >= 1) AND (corr_test.c2 <= 10000000))
- **Heap Blocks: exact=44248**
- Buffers: shared hit=71573
- -> Bitmap Index Scan on idx_corr_test_2 (cost=0.00..2842200.75 rows=10000032 width=0) (actual time=681.488..681.488 rows=9999998 loops=1)
- Index Cond: ((corr_test.c2 >= 1) AND (corr_test.c2 <= 10000000))
- **Buffers: shared hit=27325**
- Planning time: 0.147 ms
- Execution time: 2758.621 ms
- (10 rows)

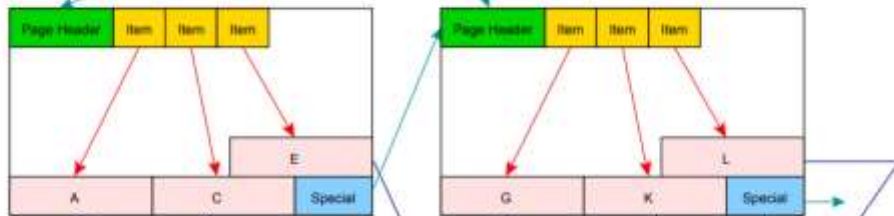
cluster

Index Page Structure

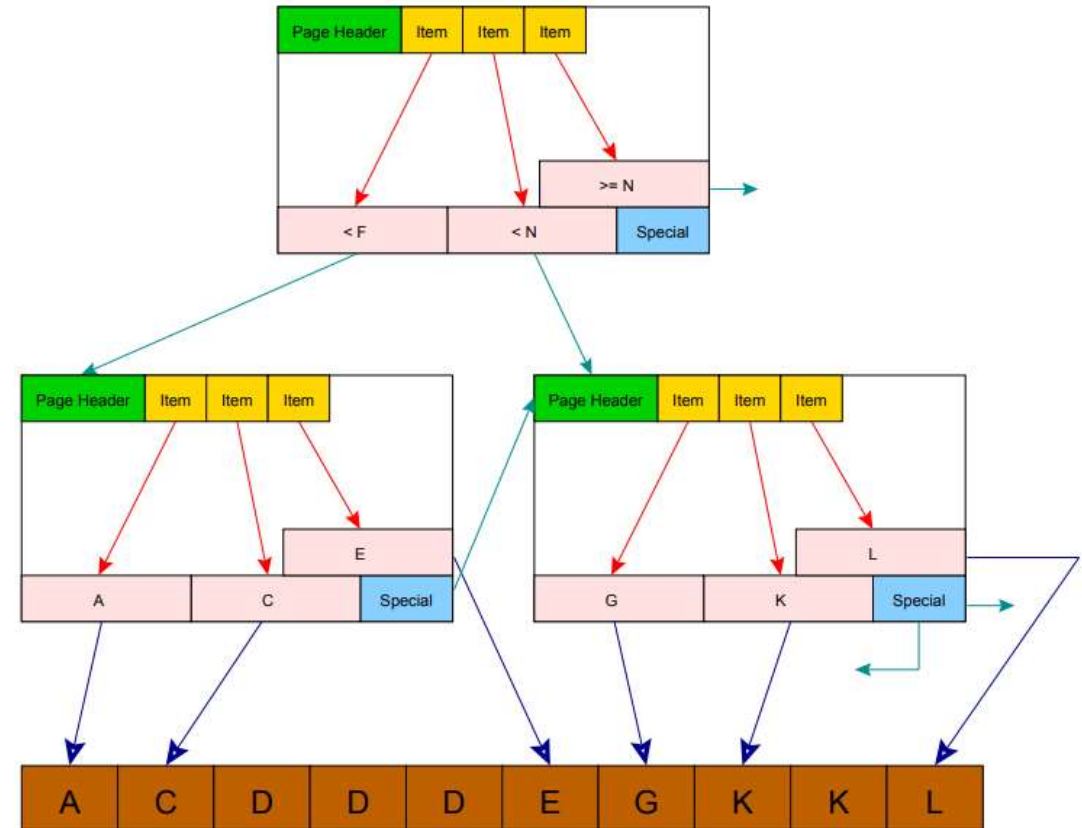
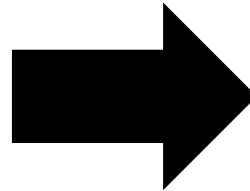
Internal



Leaf



Heap



partial index 优化

- https://github.com/digoal/blog/blob/master/201706/20170603_01.md
- WHERE 常量
- partial index
 - create index idx_tbl_1 on tbl using gist(xx, pos) where xxx > xx;

range 智能dns优化

- https://github.com/digoal/blog/blob/master/201206/20120607_01.md

导购系统优化

- 导购系统
 - 1亿历史导购文章： **数组（商品ID）相似判断**
 - 实时判定盗文
 - 毫秒级



痛点

- 多值存储和高效检索
 - 海量多值数据，相似查询，毫秒响应
 - 海量短文相似查询，毫秒响应
- 图像特征值存储和高效检索
 - 图片相似查询，毫秒响应
 - 图像识别，毫秒响应



难以加速

云产品方案、效果

- RDS PG

- smlar插件

- 相似文本、数组
 - 海明码切片(转码)相似

A handwritten table on a chalkboard illustrating the use of Hamming code slices for similarity comparison. The table has columns for 'Heap ID' and 'Key', and rows for 'Page' numbers. The 'Heap ID' column contains values 1, 35, 44, ..., 101, 109, 1000. The 'Key' column contains values 1, 3, 101, 198, 798, 10009. The 'Page' column contains values 1, 0, 0, ..., 1, 0, 0. The table shows the Hamming code slices for each 'Page' value, with the 'Key' column indicating the similarity score (e.g., 2, 1, 3, ..., 4, 2, 2).

Heap ID	Key	Page
1	1	0
3	0	0
101	0	1
198	0	1
798	1	0
10009	0	0

2 1 3 ... 4 2 2

海明码切分

```
id | hmval | hmarr
---+-----+-----
1 | 010101111111010000001001011011011000111110111101101100000011 | {1_010101111111010,2_000001001011011,3_011000111110111,4_111101111011011}
2 | 11010110000100000000000000001101101111011101110100000010101011 | {1_1101011000010000,2_000000000000110,3_110111101110111,4_0111011101000000}
3 | 010100001011011001000101010001010100100101011111011000110011 | {1_010100001011010,2_1100100010101000,3_1010100100101011,4_001010111110110}
4 | 01110001111000111110001001110000110111111000001110101101000100 | {1_011100011110001,2_1110001001110000,3_1110111111000001,4_110000011101011}
5 | 00101110101010111101001110110010011101111111001101110010011 | {1_001011101010101,2_111010011101100,3_100111101111111,4_111111100111011}
6 | 011011111001110010011001011101000000001100011000011110001010110 | {1_0110111110011100,2_1001100101110100,3_0000001110001100,4_1000110000111100}
7 | 01001101001110011100110111010011101110101001000101010110110 | {1_0100110100111001,2_1100110111101001,3_1110111010100100,4_1010010001010101}
8 | 011001011100110011100001101101110000110011111101111011010100010 | {1_0110010111001100,2_111000011011011,3_000011001111110,4_1111111011110110}
9 | 011011101011000000101010111000101110000010011100011100101000100 | {1_0110111010110000,2_0010101011110001,3_0111000001001110,4_0100111000111001}
10 | 010110100000011010010110001111111000101110001010011100110101011 | {1_0101101000000110,2_100101100011111,3_1100010111000101,4_1100010100111001}
(10 rows)
```

count (distinct) 优化 count_distinct

- HLL (估值)
 - https://github.com/digoal/blog/blob/master/201809/20180913_04.md
 - https://github.com/digoal/blog/blob/master/201302/20130226_01.md
 - https://github.com/digoal/blog/blob/master/201302/20130227_01.md
 - https://github.com/digoal/blog/blob/master/201302/20130228_01.md
- count_distinct
 - https://github.com/digoal/blog/blob/master/201903/20190317_01.md

分页优化

- https://github.com/digoal/blog/blob/master/201605/20160506_01.md
 - 分页数估算
 - 游标, 降低OFFSET损耗
 - 位点, `create table test(id int, c1 int, c2 int, c3 int, c4 int, crt_time timestamp);`
 -
 - `create index idx_test_1 on test(c1, crt_time);`
 -
 - `insert into test select id, random()*10, random()*10, random()*10, random()*10, clock_timestamp() from generate_series(1,10000000) t(id);`
 -
 - `select * from test where c1=1 and c2 between 1 and 10 order by crt_time limit 10 offset ?;`

- create table test1(pk serial8 primary key, -- 新增PK
- id int, c1 int, c2 int, c3 int, c4 int, crt_time timestamp);
-
- create index idx_test1_1 on test1(c1, crt_time, pk); -- 索引修改
-
- insert into test1 (id,c1,c2,c3,c4,crt_time) select id, random()*10, random()*10, random()*10, random()*10, clock_timestamp() from generate_series(1,10000000) t(id);
-
- select * from test1 where c1=1 and c2 between 1 and 10
- and crt_time>=? -- 位点为上一次请求的最大crt_time
- and pk>? -- 位点为上一次请求的最大pk值
- order by crt_time,pk limit 10; -- 改成order by crt_time,pk, 同时去掉offset

数据采样优化

- https://github.com/digoal/blog/blob/master/201804/20180403_03.md
- https://github.com/digoal/blog/blob/master/201709/20170911_02.md
- https://github.com/digoal/blog/blob/master/201706/20170602_02.md
- https://github.com/digoal/blog/blob/master/201609/20160929_01.md

近似统计优化

- 统计信息估算
- https://github.com/digoal/blog/blob/master/201804/20180403_03.md
- https://github.com/digoal/blog/blob/master/201509/20150919_02.md
- 估值计算(HLL)
- https://github.com/digoal/blog/blob/master/201809/20180913_04.md
- 采样估算

随机offset 优化

- 返回随机记录
- https://github.com/digoal/blog/blob/master/201810/20181009_01.md
- 随机文章推荐
- https://github.com/digoal/blog/blob/master/201808/20180810_01.md

秒杀

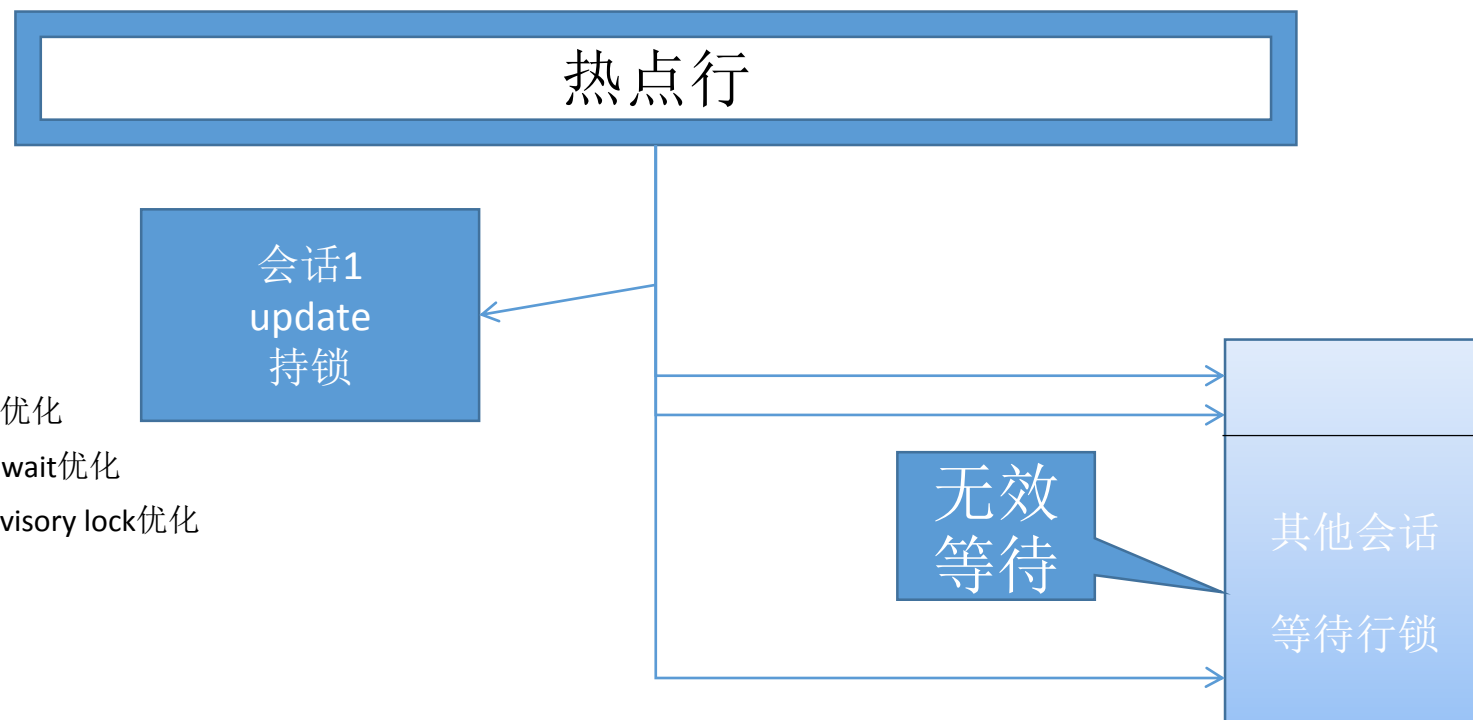
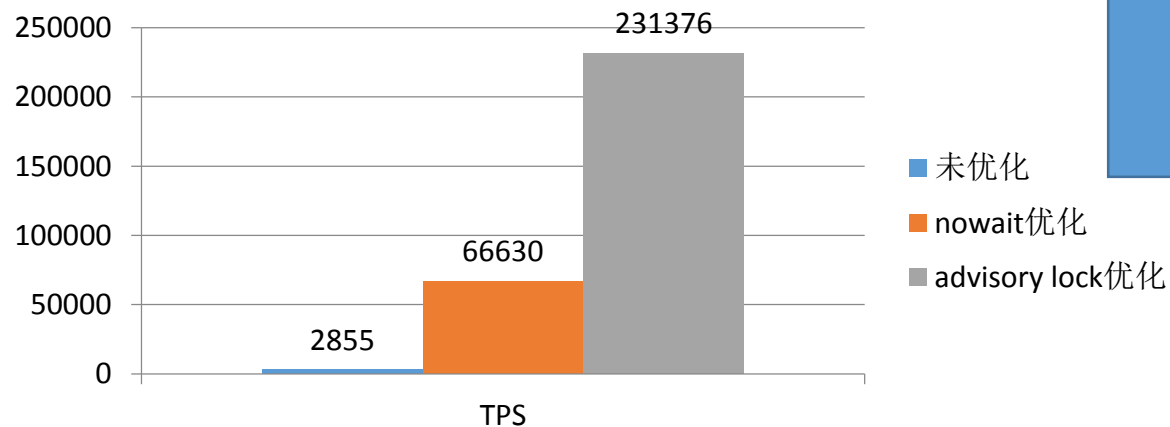
- adlock
- skip locked
- https://github.com/digoal/blog/blob/master/201801/20180105_03.md
- https://github.com/digoal/blog/blob/master/201711/20171107_31.md
- https://github.com/digoal/blog/blob/master/201509/20150914_01.md

秒杀

- 超轻锁 (advisory LOCK) 解决高并发锁竞争问题
 - 手段：在CPU运算发现行锁之前就知道是不是有冲突，大大缩短CPU计算资源，等待资源

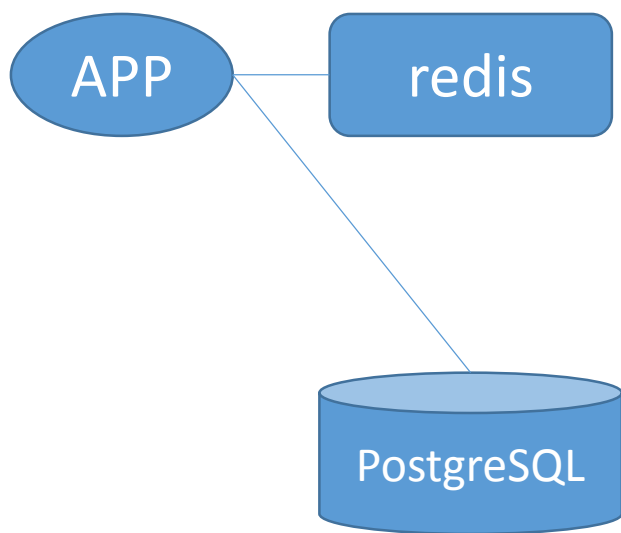
传统 - 行锁弊端

1. 无效等待多
2. 无效等待用户
长时间占用会话资源
3. 发现锁冲突的代码路径长
需要进行大量CPU运算



ADLock代替行锁 - 秒杀

- 高并发扣减库存
- 高并发争抢锁
 - `update tbl set x=x where id=? and pg_try_advisory_xact_lock(id) returning *;`



单条记录被并发更新，吞吐23万qps。

1. 连接redis判断是否还有库存
2. 有，去PG扣减(ADLock)。没有则直接返回。
3. 扣减成功，去redis更新库存

UUID IO优化

- https://github.com/digoal/blog/blob/master/201210/20121024_01.md
- https://github.com/digoal/blog/blob/master/201812/20181202_01.md

时序优化

- timescaledb
- 分区
- brin索引
- 动态封装
- https://github.com/digoal/blog/blob/master/201711/20171102_02.md
- https://github.com/digoal/blog/blob/master/201709/20170927_03.md
- https://github.com/digoal/blog/blob/master/201705/20170511_01.md
- https://github.com/digoal/blog/blob/master/201704/20170417_01.md

索引与nulls 优化

- 索引KEY值顺序
- https://github.com/digoal/blog/blob/master/201711/20171111_02.md
- https://github.com/digoal/blog/blob/master/201711/20171127_01.md

任意字段组合查询

- gin
- single index
- bitmap scan 多索引合并

高并发事务优化

- 分组提交

- `#commit_delay = 0` # range 0-100000, in microseconds
- `#commit_siblings = 5` # range 1-1000

- 异步提交

- `wal_writer_delay = 10ms` # 1-10000 milliseconds
- `#wal_writer_flush_after = 1MB` # measured in pages, 0 disables

批量写入优化 extend, vacuum

- 避免page extend lock
 - 预分配、BIG PAGE
- 避免vacuum
 - 关闭auto vacuum

json 查询优化

- gin index
 - <https://www.postgresql.org/docs/devel/functions-json.html>
- json 任意value搜索
 - https://github.com/digoal/blog/blob/master/201807/20180731_01.md
- json 任意路径搜索
 - jsonb_path_ops
- 表达式索引
 - https://github.com/digoal/blog/blob/master/201807/20180703_02.md
 - https://github.com/digoal/blog/blob/master/201807/20180702_02.md
 - https://github.com/digoal/blog/blob/master/201802/20180208_01.md

标签筛选优化 (rum, varbitx)

- https://github.com/digoal/blog/blob/master/201712/20171212_01.md

案例

- XXXpush
 - 业务背景： ToB 实时圈人系统
 - 数据来源： 实时标签数据
 - 数据规模： 单表10亿条记录， 单个B-1亿用户， 1万个标签字段。
 - 数据描述： 每个用户的标签数据
 - 查询需求： 任意标签组合圈人
 - 100毫秒级响应
 - 并发需求： 200+
 - DML需求： 实时标签分钟级体现到查询中

痛点

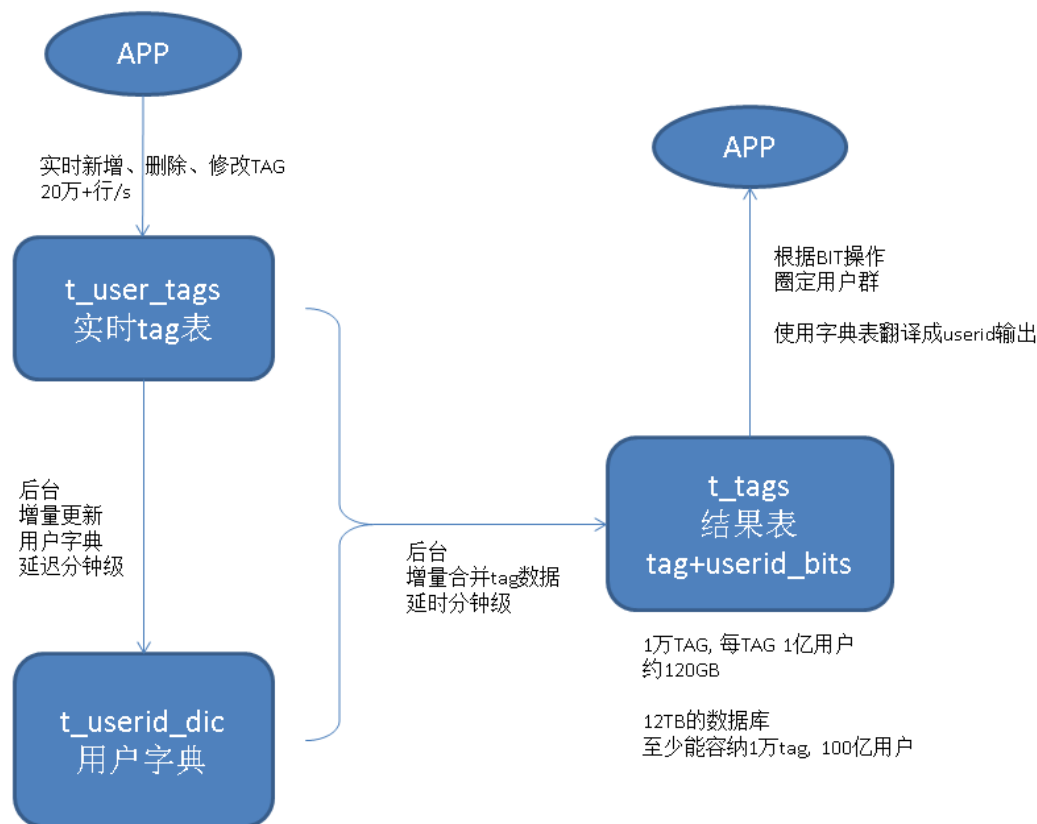
- 1万个TAG，大宽表。
 - 目前没有数据库支持。需要拆分成多表。
- 原方案成本高，收益低。
 - 8台，数据延迟**天级**别，响应时间接近**分钟级**，并发不到100。



效率低

云产品方案、效果

- RDS PG
 - 阿里云varbitx插件
 - 翻转存储 tag, userid_varbit
 - 用户ID字典化
- 单台RDS PG
 - 标签数据合并延迟**10分钟级**
 - 查询响应**毫秒级**
 - 支持并发**500+**
 - 裸空间节省80倍算上索引至少240倍节

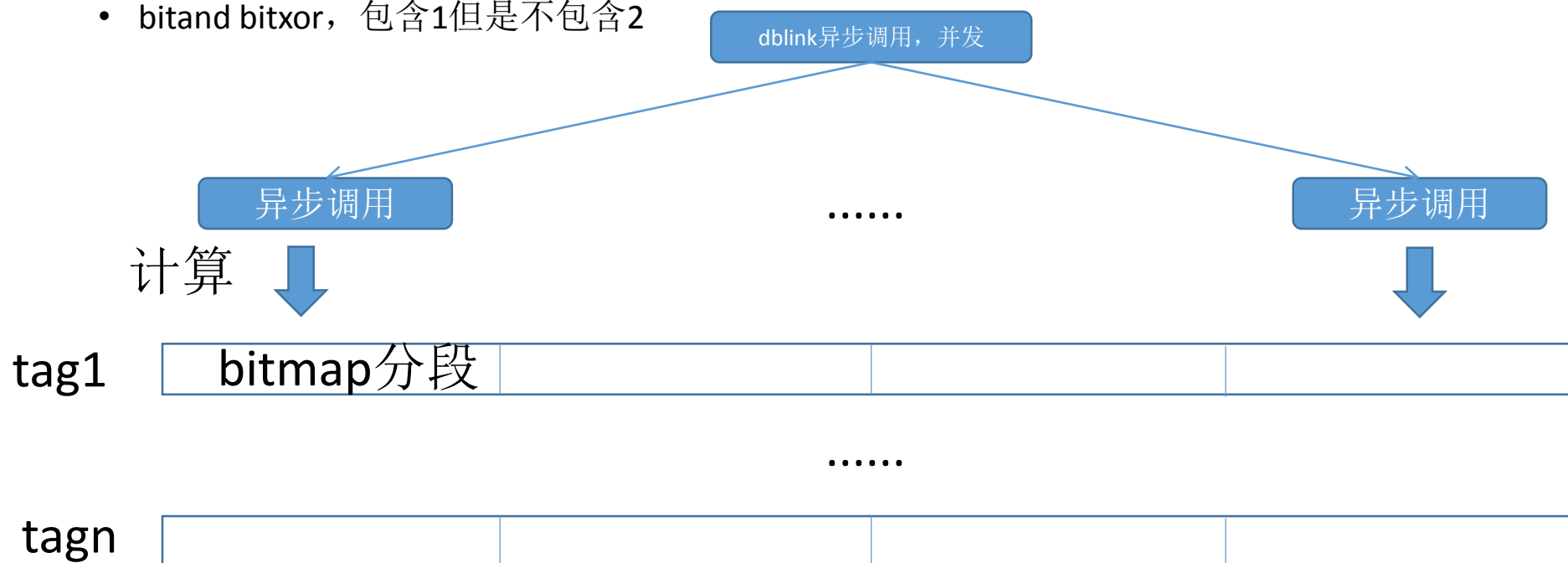


案例

- 数据银行项目(一期)
- 20亿+用户，万级标签，大屏展示（100+标签组合圈选透视）
 - 1、求COUNT，2000亿（20亿用户，100个标签组合）USER_IDS，响应速度**2.6秒**。
 - 2、求USERID明细，返回500万用户ID位置，**692毫秒**。
 - 3、求USERID明细，返回BITMAP，500万个BIT，**224毫秒**。

案例

- 数据银行项目(一期)
- 超大BIT分段，加速
- bitand，同时包含
- bitor，包含任意
- bitand bitxor，包含1但是不包含2



案例

- 数据银行项目(二期)
- HDB PG
- roaring bitmap
- https://github.com/digoal/blog/blob/master/201801/20180127_01.md
- roaring bitmap vs varbit
- 1、压缩
- 2、BITMAP操作接口
- 3、分阶段聚合

批量更新优化 (skip lock, parallel)

- https://github.com/digoal/blog/blob/master/201610/20161018_01.md
- https://github.com/digoal/blog/blob/master/sec/20180314_03.md
 - with tmp as (select t1.id,t2.info,t2.crt_time from t_dest t1 join t_batch2 t2 on (t1.id=t2.id) for update skip locked) ,
 - tmp1 as (update t_dest set info=tmp.info,crt_time=tmp.crt_time from tmp where t_dest.id=tmp.id returning t_dest.id)
 - delete from t_batch2 where exists (select 1 from tmp1 where tmp1.id=t_batch2.id);

复杂计算优化（并行，HASH，JOIN顺序，HINT）

- GEQO
- JOIN顺序固定
 - #from_collapse_limit = 8
 - #join_collapse_limit = 8 # 1 disables collapsing of explicit
- join method
 - HASH JOIN
 - merge join
 - nestloop join
- hint
 - pg_hint_plan
- sr_plan
 - 固定执行计划
- 并行计算

练习

参考